

Today 3:56 PM

Uploaded a file

Guide me

Ankush, I've reviewed your **Python/Django Hiring Assignment: AI-Assisted Box Selection System**.

This assignment is a good opportunity to demonstrate your Django skills in a practical hiring task. Since you're preparing for a fresher software developer role, I'll guide you through it step by step, focusing on understanding the problem, implementing the backend, writing tests, and preparing a professional GitHub submission.

The assignment asks you to build a Django-based system that recommends a suitable shipping box for an order based on product dimensions, weight, box capacity, and cost.

Assignment - Python_Django Deve...

1. Understand what you need to build

Imagine an ecommerce warehouse receives an order containing three products.

Each product has:

- Length
- Width
- Height
- Weight

The warehouse has several shipping boxes, each with:

- Internal length, width, and height
- Maximum weight capacity
- Cost

Your system must determine which box is suitable for the order.

For example:

Box	Internal dimensions	Max weight	Cost
Small	20 × 15 × 10 cm	2 kg	₹20
Medium	30 × 25 × 20 cm	5 kg	₹35

Box	Internal dimensions	Max weight	Cost
Large	50 × 40 × 30 cm	15 kg	₹60

If an order fits inside the Small box and satisfies its weight limit, the system might recommend it rather than a more expensive box.

However, we must establish the precise selection rules before implementing the recommendation logic. We shouldn't assume that the cheapest box is always the correct answer without considering the assignment's requirements.

2. The development plan

We'll divide the assignment into manageable stages.

Stage 1: Analyze the requirements

Before writing code, answer these questions:

1. How will the system store products?
2. How will it store boxes?
3. How will it represent an order containing multiple products?
4. How will it determine whether an order fits inside a box?
5. How will it handle orders that cannot fit into any available box?
6. If multiple boxes are suitable, how will it select one?

Your first task: Think about these questions. You don't need to know all the answers immediately.

Stage 2: Design the Django models

We'll identify the models needed for the application.

A possible starting point is:

Model	Purpose
Product	Stores product dimensions and weight
Box	Stores box dimensions, weight capacity, and cost
Order	Represents a customer's order
OrderItem	Connects products to an order and records quantities

These are preliminary suggestions, not confirmed requirements. We'll decide whether all four models are necessary after examining the expected functionality.

Stage 3: Implement the box-selection algorithm

This is the central part of the assignment.

We'll work through the logic in Python before integrating it into Django.

The algorithm will need to consider:

- Whether the products physically fit inside a box.
- Whether the combined weight is within the box's maximum capacity.
- How to choose between multiple suitable boxes.
- What happens when no box is suitable.

We'll also discuss an important detail: **whether products can be rotated and how multiple products can be packed together**. Simple dimension comparisons do not necessarily prove that several products can physically fit inside a box.

We'll avoid implementing a complicated packing algorithm unless the requirements justify it.

Stage 4: Build the Django application

Once the logic is clear, we'll implement the application.

We'll work through:

1. Creating the Django project and app.
2. Creating the models and migrations.
3. Adding products and boxes.
4. Implementing the recommendation functionality.
5. Creating the necessary views and URLs.
6. Providing a way to submit an order and see its recommended box.

We'll decide whether a traditional Django interface or a REST API is more appropriate based on the assignment's actual requirements.

Stage 5: Write test cases

The assignment explicitly requires test cases and test-run output.

Assignment - Python_Django Deve...

We'll test scenarios such as:

Test scenario	Expected behavior
Order fits inside a suitable box	Recommend a suitable box
Order exceeds a box's weight capacity	Reject that box
Product exceeds the box's dimensions	Reject that box
Order cannot fit into any available box	Handle the situation appropriately
Multiple boxes can accommodate an order	Apply the defined selection rule
Order contains multiple products	Evaluate the complete order

These are proposed test scenarios. We'll refine them once the selection rules are established.

Stage 6: Prepare your GitHub submission

The assignment requires the following deliverables:

Deliverable	What we'll do
GitHub repository	Organize and upload the project
README.md	Explain the application, setup, and usage
AI_USAGE.md	Document your actual AI usage
Chat transcript	Export and include your actual conversation
Learning reflection	Describe what you personally learned
Test cases	Include automated tests
TEST_OUTPUT.md or GitHub Actions	Provide evidence of test execution

The assignment specifically states that the chat transcript must not be generated using AI and that the learning reflection must be your own work. We'll respect those requirements. I can help you understand the work and organize your submission, but you should produce those two items yourself.

Assignment - Python_Django Deve...

3. How I recommend we work together

I don't want to simply give you a complete Django project that you cannot explain in an interview.

Instead, we'll use this approach:

1. I'll explain the requirement.
2. I'll help you design the solution.
3. You'll attempt the implementation.
4. I'll review your code and explain any mistakes.
5. We'll test the behavior.
6. You'll document the decisions you actually made.

This will help you prepare for questions such as:

- Why did you choose these models?
- How does your recommendation algorithm work?
- How do you determine whether an order fits inside a box?
- How did you handle edge cases?
- How did you test your application?
- Which parts of your implementation did you modify after reviewing AI-generated suggestions?

4. Let's start with Stage 1

Don't start coding yet. First, I want to understand how you approach the problem.

Try answering these three questions in your own words:

Question 1: Suppose a product measures $10 \times 8 \times 5$ cm, weighs 1 kg, and a box measures $15 \times 10 \times 8$ cm with a maximum weight capacity of 2 kg.

Would the product fit inside the box? Why?

Question 2: Suppose an order contains two products, each weighing 1.5 kg. The box has a maximum weight capacity of 2 kg.

Should the system accept or reject this box? Explain your reasoning.

Question 3: Suppose an order fits inside three different boxes, with costs of ₹20, ₹35, and ₹50.

How would you decide which box to recommend? Explain your reasoning.

Reply with your answers to Questions 1, 2, and 3. I'll review your reasoning, clarify anything necessary, and then guide you through designing the Django models.

1. Product fits the box as box measurements are larger than a product
2. System should reject the box as products weight limit exceeds maximum weight capacity of box
3. I would recommend the box 1 as its cheaper

Good work, Ankush! **Your answers to all three questions are correct under the assumptions we've established.** Let's review them carefully.

1. Review your answers

Question 1: Does the product fit inside the box?

Your answer: Yes, because the box measurements are larger than the product.

Correct.

- Product: $10 \times 8 \times 5$ cm
- Box: $15 \times 10 \times 8$ cm

Each product dimension is less than or equal to the corresponding box dimension.

For this simple example, the product fits without rotation.

One detail to remember: in a real packing system, we may also need to consider product rotation and whether dimensions can be rearranged.

Question 2: Should the box be rejected?

Your answer: Yes, because the combined product weight exceeds the box's maximum capacity.

Correct.

- Product 1: 1.5 kg
- Product 2: 1.5 kg
- Total: 3 kg
- Box capacity: 2 kg

Since 3 kg exceeds 2 kg, the box must be rejected.

In Python, the basic check would be:

Python

```
if total_weight > box.max_weight:  
    # Reject the box
```

Question 3: Which box should be recommended?

Your answer: Recommend the first box because it is cheaper.

Correct, assuming all three boxes satisfy the physical dimensions and weight requirements, and our selection rule is to minimize box cost.

This gives us a preliminary decision rule:

1. Reject boxes that cannot accommodate the order.
2. Among the remaining boxes, recommend the least expensive one.

We'll verify whether this is the intended selection rule before finalizing the implementation.

2. Now let's design the Django models

Before writing the models, let's identify the information our application needs to store.

We'll start with four possible models.

Model 1: Product

A product needs to store:

Field	Example
name	Laptop
length	30 cm
width	20 cm
height	5 cm
weight	1.5 kg

Model 2: Box

A box needs to store:

Field	Example
name	Medium Box
length	40 cm
width	30 cm
height	20 cm
max_weight	10 kg
cost	₹35

Model 3: Order

An order represents a customer's purchase.

For example:

- Order ID: 101
- Customer: Ankush
- Products: Laptop, Mouse

Model 4: OrderItem

Why do we need this model?

Imagine a customer orders:

- 2 laptops
- 3 mice

We need to record both the product and its quantity.

An `OrderItem` model can represent that relationship.

For example:

Order	Product	Quantity
101	Laptop	2
101	Mouse	3

This also lets us calculate the total weight of the order.

3. Your next exercise

I want you to attempt the model design yourself.

You have already studied Django models, relationships, and migrations, so this is a good opportunity to apply those skills.

Task: Design the `Product` model.

Create a Django model with these fields:

1. `name` — a character field.
2. `length` — a field for the product's length.
3. `width` — a field for its width.
4. `height` — a field for its height.
5. `weight` — a field for its weight.

Requirements

- Use appropriate Django field types.
- Choose reasonable `max_digits` and `decimal_places` values for measurements.
- Give the fields appropriate maximum lengths or other constraints where necessary.
- Use clear, readable field names.

You don't need to worry about the box-selection algorithm yet.

Write the `Product` model in your next message. I'll review your code, explain any corrections, and then we'll move on to the `Box` model.

I have written the Product model

Great, Ankush! 👍

Now **paste your `Product` model code here.**

I'll review it for:

1. Correct Django field types.
2. Appropriate `max_digits` and `decimal_places` for dimensions and weight.
3. Validation and constraints.
4. Whether the model is suitable for our box-selection algorithm.
5. Any improvements needed for a professional assignment submission.

I'll explain any corrections rather than simply replacing your code.

Once the `Product` model is ready, we'll move on to the `Box` model.

You can proceed.

Alright, Ankush. We'll proceed to the next stage of your Python/Django hiring assignment.

Since you haven't pasted your `Product` model, I'll assume you've written it and continue with the next exercise. We can review your implementation later.

Stage 2: Create the Box model

1. Understand the requirements

According to the assignment, each box has internal dimensions, a maximum weight capacity, and a cost.

Assignment - Python_Django Deve...

Our `Box` model therefore needs to store the following information:

Field	Example	Purpose
<code>name</code>	Small Box	Identifies the box
<code>length</code>	20 cm	Internal length
<code>width</code>	15 cm	Internal width
<code>height</code>	10 cm	Internal height
<code>max_weight</code>	2 kg	Maximum supported weight
<code>cost</code>	20.00	Cost of using the box

We'll use consistent units throughout the application:

- Dimensions: centimetres (cm).
- Weight: kilograms (kg).
- Cost: a decimal monetary value.

These are implementation choices for our project; the assignment itself doesn't prescribe particular units.

2. Your exercise: Implement the Box model

Create the `Box` model in your Django app.

Use the following requirements:

1. `name` — character field.
2. `length` — decimal field.
3. `width` — decimal field.
4. `height` — decimal field.
5. `max_weight` — decimal field.
6. `cost` — decimal field.

Hints

You can use `DecimalField` for dimensions, weight, and cost.

For example:

Python

```
models.DecimalField(max_digits=10, decimal_places=2)
```

Think about these questions while implementing it:

- Should the dimensions be allowed to be zero?
- Should a box have a negative cost?
- Should the box name be unique?
- Should you add a `__str__()` method?

You can use Django validators to prevent invalid values.

Try implementing the model yourself before looking for a solution. This is a hiring assignment, so understanding and explaining your decisions matters more than typing code quickly.

3. What comes next?

After completing `Box`, we'll move on to:

Stage 3: Order and OrderItem models

We'll discuss how to represent an order containing multiple products and how to calculate its total weight.

Then we'll implement the most important part of the assignment: the box recommendation algorithm.

Your next step

Write the `Box` model and paste it here. I'll review it before we proceed to the order models.

Proceed

Stage 3: Create the Order and OrderItem models

Ankush, let's continue with the box-selection assignment.

So far, we've discussed the `Product` and `Box` models. Now we need to represent customer orders.

The important requirement is that **an order can contain multiple products, and each product can have a quantity greater than one.**

For example, an order might contain:

Product	Quantity	Weight per item
Laptop	2	1.5 kg
Mouse	3	0.2 kg

The total weight is:

- Laptops: $2 \times 1.5 = 3$ kg
- Mice: $3 \times 0.2 = 0.6$ kg
- **Total order weight: 3.6 kg**

We'll need this information when checking whether a box can accommodate the order.

1. Order model

The `Order` model represents a customer's order.

For our initial design, it can contain:

- An automatically generated primary key.
- A creation timestamp.

We don't need to add customer authentication or a complete ecommerce checkout system unless the assignment requires it.

2. OrderItem model

The `OrderItem` model connects an order to a product.

We need three important pieces of information:

Field	Purpose
<code>order</code>	Identifies which order contains the product
<code>product</code>	Identifies the product being ordered
<code>quantity</code>	Records how many units of that product were ordered

Why use a separate model?

Suppose an order contains three different products.

Instead of adding fields such as `product1`, `product2`, and `product3` to the `Order` model, we can create multiple `OrderItem` records associated with the same order.

This is a more flexible design because the order can contain a varying number of products.

A `ForeignKey` is suitable for both relationships:

- One order can have multiple order items.
- One product can appear in multiple order items.

The `OrderItem` model therefore provides the connection between orders and products.

3. Your implementation task

Now create the two models yourself.

Requirements

Order

- Add an automatically generated primary key.
- Add a creation timestamp.

OrderItem

- Add a `ForeignKey` to `Order`.
- Add a `ForeignKey` to `Product`.

- Add a quantity field that stores whole numbers.
- Ensure the quantity cannot be zero or negative.

You may use Django's `PositiveIntegerField` for quantity.

Think about whether you need a `related_name` on the foreign keys. It can make accessing related objects more readable.

Before moving forward

Once you've written both models:

1. Save your changes.
2. Run `python manage.py makemigrations`.
3. Run `python manage.py migrate`.
4. Check that the migrations complete successfully.

Don't worry about creating views or templates yet.

4. What we'll do next

After the models are ready, we'll implement the **box recommendation algorithm**.

We'll start with the basic rules:

1. Calculate the total weight of the order.
2. Identify boxes that satisfy the weight requirement.
3. Check whether the products fit within the internal dimensions.
4. Select the least expensive suitable box, if that is the selection rule we establish.
5. Handle the situation where no suitable box exists.

One important limitation: the assignment doesn't specify exactly how multiple products must be packed, whether rotation is permitted, or how to handle combinations that cannot fit despite satisfying individual dimension checks. We'll need to make those decisions explicit rather than silently assume a complete packing algorithm.

Your next step: Implement `Order` and `OrderItem`, then tell me when you're ready. We'll tackle the recommendation logic together.

created order and orderItem models

Nice work, Ankush! Now that you've created the `Product`, `Box`, `Order`, and `OrderItem` models, we can move to the **most important part of this assignment: the box recommendation algorithm**.

We'll build it in small steps so you understand the logic and can explain it during an interview.

Stage 4: Box recommendation algorithm

1. Understand the problem

Suppose an order contains:

Product	Dimensions (cm)	Weight	Quantity
Laptop	30 × 20 × 5	1.5 kg	1
Mouse	10 × 5 × 3	0.2 kg	2

The total weight is:

$$1.5 + (0.2 \times 2) = 1.9 \text{ kg}$$

Now suppose the warehouse has these boxes:

Box	Dimensions (cm)	Maximum weight	Cost
Small	35 × 25 × 10	2 kg	₹20
Medium	40 × 30 × 20	5 kg	₹35
Large	50 × 40 × 30	10 kg	₹60

The algorithm needs to determine which boxes can accommodate the order and then recommend one according to our selection rule.

For now, we'll assume the objective is to select the **least expensive box that can accommodate the entire order**.

2. First, calculate the total order weight

Before checking the dimensions, we can implement a function that calculates the total weight.

The formula is:

$$\text{Total Weight} = \sum (\text{Product Weight} \times \text{Quantity})$$

For example:

- Laptop: $1.5 \times 1 = 1.5$ kg
- Mouse: $0.2 \times 2 = 0.4$ kg
- Total: 1.9 kg

In Django, your `OrderItem` records give you access to each product's weight and quantity.

Your exercise

Write a method on your `Order` model called `get_total_weight()`.

It should:

1. Retrieve the order's related items.
2. Multiply each item's product weight by its quantity.
3. Add the results together.
4. Return the total weight.

Hint: You can use a loop to iterate over the related `OrderItem` objects.

If your foreign key uses a custom `related_name`, use that name to access the items.

Otherwise, Django provides a default reverse relationship name.

3. Test your method

Once you've implemented `get_total_weight()`, test it with these cases:

Test	Products	Expected total
1	1 laptop (1.5 kg)	1.5 kg
2	2 laptops (1.5 kg each)	3.0 kg
3	1 laptop (1.5 kg) + 2 mice (0.2 kg each)	1.9 kg
4	An order with no items	0 kg

Use `Decimal` values consistently if your model's weight field is a `DecimalField`.

What comes after this?

Once the weight calculation works, we'll implement the box filtering step:

- Reject boxes whose maximum weight is less than the order's total weight.

- Check the dimensional constraints.
- Compare the costs of the remaining suitable boxes.

For now, implement `get_total_weight()` and paste your code here. I'll review it before we proceed to the dimensional checks.

```
def get_total_weight(self):  
  
    total = Decimal("0")  
  
    for item in self.items.select_related("product"):  
  
        total += item.product.weight * item.quantity  
  
    return total
```

Review of your `get_total_weight()` method

Ankush, **your implementation is correct.** 👍

You've used a good approach for calculating the total weight of an order.

Here's what you did well:

1. Initialized the total using `Decimal("0")`, which is appropriate when working with Django `DecimalField` values.
2. Iterated over the order's related items.
3. Used `select_related("product")` to retrieve the associated product efficiently.
4. Multiplied each product's weight by its quantity.
5. Returned the accumulated total.

Your use of `self.items` also assumes that your `OrderItem` foreign key has `related_name="items"`. If that's what you defined in your model, this is correct.

One thing to verify: make sure you've imported `Decimal` from Python's `decimal` module.

Stage 5: Check whether a product fits inside a box

Now we'll implement the dimensional check.

1. Understand the basic rule

A product fits inside a box if its dimensions can fit within the box's internal dimensions.

For example:

Product: $10 \times 8 \times 5$ cm

Box: $15 \times 10 \times 8$ cm

The product fits.

However, consider this:

Product: $10 \times 8 \times 5$ cm

Box: $8 \times 10 \times 15$ cm

The product can still fit if rotation is allowed.

To account for rotation, we can sort the three dimensions of both objects and compare them.

For example:

Python

```
product_dimensions = sorted([10, 8, 5])
box_dimensions = sorted([8, 10, 15])
```

The sorted dimensions are:

```
Product: [5, 8, 10]
Box:      [8, 10, 15]
```

Each product dimension is less than or equal to its corresponding box dimension.

Important limitation: This comparison checks whether an individual product fits in a box in some orientation. It does not prove that multiple products can be packed together without overlapping. We'll need to account for that limitation when designing the complete recommendation system.

2. Your exercise

Write a method called `fits_in_box()` that accepts a product and a box.

It should:

1. Collect the product's length, width, and height.

2. Collect the box's internal length, width, and height.
3. Sort both sets of dimensions.
4. Compare the corresponding dimensions.
5. Return `True` if the product fits, otherwise return `False`.

You can use Python's `sorted()` function and `all()`.

Hint

The comparison you need is:

Python

```
all(product_dim <= box_dim for product_dim, box_dim in ...)
```

You'll need to decide how to construct the pairs of dimensions.

Try implementing `fits_in_box()` **yourself and paste your code here.** I'll review it, then we'll combine the dimensional and weight checks into the recommendation algorithm.

next

Stage 6: Combine the checks and recommend a box

Ankush, let's move forward.

We've already discussed how to calculate the total order weight and how to check whether an individual product fits inside a box.

Now we'll combine those ideas into a **box recommendation algorithm**.

The assignment requires a system that recommends a suitable shipping box based on product dimensions, weight, box capacity, and cost.

Assignment - Python_Django Deve...

1. Define the selection rules

For our initial implementation, we'll use these rules:

1. An order must contain products.
2. The box must satisfy the order's total weight requirement.
3. Every product must fit within the box's internal dimensions, allowing rotation.
4. Among the boxes that pass these checks, select the one with the lowest cost.
5. If no box passes, return `None`.

Important: Checking each product individually does not guarantee that multiple products physically fit together in a box. We'll treat this as a simplified first version and clearly document the limitation.

2. Understand the algorithm

Imagine the warehouse has 100 boxes.

We don't want to recommend a box simply because it is cheap. First, we must determine whether it can accommodate the order.

The process looks like this:

```
graph TD
    Start([Start]) --> V1[v]
    V1 --> Calculate[Calculate total weight]
    Calculate --> V2[v]
    V2 --> Get[Get available boxes]
    Get --> V3[v]
    V3 --> CheckBox[Check each box's capacity]
    CheckBox --> V4[v]
    V4 --> CheckProd[Check product dimensions]
    CheckProd --> V5[v]
    V5 --> Keep[Keep only suitable boxes]
    Keep --> V6[v]
    V6 --> Select[Select the cheapest box]
    Select --> V7[v]
    V7 --> Return([Return recommended box])
```

If no suitable boxes remain, return `None`.

3. Your implementation exercise

Create a method called `recommend_box()`.

You can implement it as a method on your `Order` model or as a separate function in a suitable module.

Requirements

Your method should:

Step 1: Calculate the order's total weight using your existing `get_total_weight()` method.

Step 2: Retrieve the available boxes.

Step 3: Iterate through the boxes.

Step 4: Reject a box if its maximum weight capacity is less than the total order weight.

Step 5: For the remaining boxes, check whether every product fits inside the box.

Step 6: Collect the suitable boxes.

Step 7: Return the suitable box with the lowest cost.

Step 8: If no suitable box exists, return `None`.

Hints

You may find these Python concepts useful:

- `for` loops
- `if` conditions
- `all()`
- `sorted()`
- Django QuerySets
- `order_by()`

For the cost comparison, remember that Django QuerySets can be ordered by a model field.

Also, be consistent with `Decimal` values when comparing weights and costs.

Stage 7: Test the recommendation algorithm

Before connecting the algorithm to views or templates, we need to verify that it behaves correctly.

The following examples use the simplified packing rules we've established.

Scenario	Expected result
One product fits in multiple boxes	Select the cheapest suitable box

Scenario	Expected result
Product is too large for a box	Reject that box
Order exceeds the box's weight capacity	Reject that box
Several boxes are suitable	Select the least expensive
No boxes are suitable	Return <code>None</code>
Order contains no products	Return <code>None</code>

Exercise 1: Weight capacity

A product weighs 3 kg.

Two boxes are available:

- Box A: capacity 2 kg, cost ₹15.
- Box B: capacity 5 kg, cost ₹30.

Assuming the product fits dimensionally in both boxes, which box should be recommended?

Exercise 2: Cost

An order fits in three boxes:

- Box A: ₹50.
- Box B: ₹30.
- Box C: ₹40.

Which box should be recommended?

Exercise 3: No suitable box

An order weighs 8 kg, but the available boxes have maximum capacities of 2 kg, 5 kg, and 7 kg.

What should the algorithm return?

Stage 8: Integrate the algorithm into Django

Once your recommendation function is working, the next step is to make it accessible through your Django application.

A simple implementation could use:

- A Django view to receive an order ID.
- The recommendation function to determine the suitable box.
- A response that displays the recommended box's name, dimensions, and cost.

For example, the result might look like:

```
Order: 101
```

```
Recommended Box: Medium Box
```

```
Dimensions: 40 × 30 × 20 cm
```

```
Maximum Weight: 5 kg
```

```
Cost: ₹35.00
```

We can implement this using a traditional Django view or a Django REST Framework API. The assignment doesn't explicitly require either approach, so we'll choose based on the functionality you want to demonstrate.

Answers to the exercises

1. **Box B** — Box A cannot support the product's 3 kg weight.
2. **Box B** — It is the least expensive suitable box.
3. **Return** `None` — None of the boxes can support the order's 8 kg weight.

Your next step

Before we move into views, let's establish where your implementation stands.

Have you written the `fits_in_box()` **and** `recommend_box()` **functions yet?**

If you have, paste your code here and I'll review it. If you haven't, I can walk you through implementing them one at a time.

Next

Stage 9: Build the Django view

Ankush, let's continue with your hiring assignment.

We've discussed the models and the box recommendation logic. Now we'll connect that logic to Django so someone can submit an order ID and see the recommended shipping box.

We'll start with a **traditional Django view**. This keeps the implementation straightforward and lets you demonstrate your existing Django knowledge.

The workflow will be:

1. Receive an order ID.
2. Retrieve the corresponding order.
3. Run the recommendation algorithm.
4. Display the recommended box.
5. Handle the situation where no suitable box is found.

1. Create the view

Open your app's `views.py`.

We'll use Django's `get_object_or_404()` to retrieve the order. If the order doesn't exist, Django will return a 404 response.

Here is the basic structure:

Python

```
from django.shortcuts import get_object_or_404, render

from .models import Order


def recommend_box_view(request, order_id):
    order = get_object_or_404(Order, id=order_id)

    recommended_box = order.recommend_box()

    context = {
        "order": order,
        "recommended_box": recommended_box,
    }

    return render(request, "recommendation/result.html", context)
```

This assumes your recommendation method is named `recommend_box()` and is defined on `Order`. If you implemented it as a separate function, you'll need to call that function instead.

Understand each part

1. Retrieve the order

Python

```
order = get_object_or_404(Order, id=order_id)
```

This retrieves the order with the specified ID.

2. Run the recommendation

Python

```
recommended_box = order.recommend_box()
```

This calls your recommendation logic.

3. Prepare the context

Python

```
context = {  
    "order": order,  
    "recommended_box": recommended_box,  
}
```

The context makes the order and recommendation available to the template.

4. Render the template

Python

```
return render(request, "recommendation/result.html", context)
```

Django renders the template and returns the resulting HTML response.

2. Add the URL

Open your app's `urls.py`.

Add a URL pattern that accepts an order ID:

Python

```
from django.urls import path  
  
from . import views  
  
urlpatterns = [  
    path(  
        #  
    )  
]
```

```
        "orders/<int:order_id>/recommend/",
        views.recommend_box_view,
        name="recommend_box",
    ),
]
```

Make sure your project's main `urls.py` includes your app's URL configuration.

For example, if your app is named `shipping`, you might include it using:

Python

```
path("", include("shipping.urls"))
```

You'll also need to import `include` from `django.urls`.

You can then access a recommendation using a URL such as:

```
http://127.0.0.1:8000/orders/1/recommend/
```

Here, `1` is the order ID.

3. Create the result template

Create this directory structure inside your app:

```
your_app/
  templates/
    recommendation/
      result.html
```

In `result.html`, display the order and the recommendation.

Your template should handle two situations:

- A suitable box was found.
- No suitable box was found.

You can use Django's template conditionals:

django

```
{% if recommended_box %}
    <!-- Display the recommended box here -->
{% else %}
```

```
<!-- Display a message that no suitable box was found -->
{% endif %}
```

Inside the successful branch, display the box's name, dimensions, and cost.

Use the actual field names from your `Box` model.

Stage 10: Add automated tests

The assignment explicitly asks for test cases and test-run output.

Assignment - Python Django Deve...

We'll use Django's built-in testing framework.

1. What should we test?

At a minimum, test the following:

Test	Expected behavior
Existing order with a suitable box	Recommendation is displayed
Existing order with no suitable box	No-suitable-box message is displayed
Nonexistent order ID	Returns HTTP 404
Order exceeds every box's weight capacity	No suitable box is recommended
Product exceeds available box dimensions	No suitable box is recommended

2. Test the view

Create tests in your app's `tests.py`.

For the view, Django's test client can make requests without starting a live server.

For example, the general structure is:

Python

```
from django.test import TestCase
from django.urls import reverse

class RecommendationViewTests(TestCase):

    def test_nonexistent_order_returns_404(self):
        response = self.client.get(
            reverse("recommend_box", args=[9999])
```

```
)  
  
self.assertEqual(response.status_code, 404)
```

This test assumes that order ID `9999` does not exist in the test database.

Notice that we use `reverse()` instead of hardcoding the URL. This makes the test easier to maintain if the URL pattern changes.

Stage 11: Run your tests

Open your terminal in the directory containing `manage.py`.

Run:

Bash

```
python manage.py test
```

Django will discover and execute your tests.

A successful run should produce output similar to:

```
Ran 5 tests in 0.XXXs
```

```
OK
```

This is an illustrative example, not your actual test output. Include the output from your own test run in `TEST_OUTPUT.md`.

Stage 12: Prepare your README.md

Your README should explain:

1. **Project title:** AI-Assisted Box Selection System.
2. **Overview:** What the application does.
3. **Features:** Product management, box management, order handling, and box recommendations.
4. **Technologies:** Python, Django, and your database.
5. **Installation instructions:** How someone can set up the project.
6. **Usage:** How to create products, boxes, and orders and obtain a recommendation.
7. **Testing:** How to run the automated tests.

8. Limitations: Explain any assumptions made by your packing algorithm.

Be accurate about the features you've actually implemented. Don't claim that the application supports advanced 3D packing if it only checks individual product dimensions.

Your remaining submission checklist

Item	Status
Product model	Created according to your earlier update
Box model	Previously discussed; implementation not reviewed
Order and OrderItem models	Created
Total-weight calculation	Code reviewed and accepted
Dimensional fitting function	Not yet reviewed
Recommendation algorithm	Not yet reviewed
Django view and template	Implementation not confirmed
Automated tests	To be completed and executed
README.md	To be completed
AI_USAGE.md	Must document your actual AI use
Chat transcript	Export the actual conversation yourself
Learning reflection	Write this yourself
TEST_OUTPUT.md or GitHub Actions	Include actual test-run evidence

The assignment specifically requires the chat transcript and learning reflection to be your own work, so don't have AI generate either of those deliverables.

Assignment - Python_Django Deve...

What I want you to do now

Let's pause adding new features and verify the implementation.

Please tell me which of these describes your current progress:

A. I have implemented `fits_in_box()` and `recommend_box()`.

B. I have implemented `fits_in_box()`, but not `recommend_box()`.

C. I haven't implemented either function yet.

D. I have implemented both functions, the view, and the template.

Reply with **A, B, C, or D**. I'll use your answer to choose the next step rather than making you repeat work you've already done.

I have implemented everything

Excellent, Ankush! If you've implemented the models, weight calculation, dimensional checks, recommendation algorithm, view, and template, **you're ready to move from implementation to verification and submission preparation.**

For a hiring assignment, getting the code to run is only part of the job. We also need to verify edge cases, demonstrate that the tests pass, and make sure you can explain your design decisions.

We'll proceed in this order:

1. Verify the recommendation algorithm.
2. Run and strengthen the automated tests.
3. Check the Django application for errors.
4. Prepare the required documentation.
5. Review the GitHub repository before submission.
6. Practice explaining your project in an interview.

Stage 13: Verify your complete application

Step 1: Run Django's system checks

In the terminal, from the directory containing `manage.py`, run:

Bash

```
python manage.py check
```

This checks for common problems in your Django project.

Then run:

Bash

```
python manage.py test
```

Make sure the tests complete successfully.

Step 2: Test the recommendation logic manually

Use the following scenarios to check your implementation.

Scenario	Expected result
Product fits in several boxes	Cheapest suitable box is recommended
Product exceeds a box's weight capacity	That box is rejected
Product dimensions exceed the box dimensions	That box is rejected
Order contains multiple products	Total weight accounts for every item's quantity
No available box can accommodate the order	No suitable box is returned
Order ID does not exist	The view returns a 404 response

Also check the boundary cases:

- Product dimensions exactly equal the box dimensions.
- Total order weight exactly equals the box's maximum capacity.
- Two suitable boxes have the same cost.
- The order contains no items.

For the last two cases, make sure your behavior is intentional and consistent with your documented rules.

Stage 14: Check your assignment requirements

Your assignment requires the following submission items.

Assignment - Python_Django Deve...

File or item	What you need to do
GitHub repository	Upload your completed project
README.md	Explain the project, setup, and usage
AI_USAGE.md	Document the AI tools, prompts, accepted/rejected output, mistakes, and verification

File or item	What you need to do
Chat transcript	Export and include the actual conversation yourself
Learning reflection	Write your own reflection about what you learned
Test cases	Include your automated tests
<code>TEST_OUTPUT.md</code>	Include the actual test output, or provide a GitHub Actions link

Important: Your `AI_USAGE.md` should accurately describe how you used AI, including the assistance you received during this conversation. Don't claim that you independently wrote or verified something if you didn't.

Stage 15: Prepare to explain your project

You should be able to explain these five things without reading your code:

1. **Models:** Why did you use separate `Order` and `OrderItem` models?
2. **Weight calculation:** How does `get_total_weight()` calculate the total?
3. **Dimensional check:** How does your implementation determine whether a product fits inside a box?
4. **Recommendation:** How does your algorithm choose between suitable boxes?
5. **Testing:** What edge cases did you test, and what did the results demonstrate?

One important technical point to be ready to discuss: checking each product individually is not necessarily sufficient to prove that all products physically fit together in one box. Be prepared to explain the limitations of your implementation and any assumptions you made.

Your next task

Let's verify the actual application rather than assuming everything works just because the implementation is complete.

Run these two commands in your project terminal:

Bash

```
python manage.py check
```

Bash

```
python manage.py test
```

Then send me:

1. The output of `python manage.py check`.
2. The output of `python manage.py test`.
3. Any errors you encounter.

I'll review the results and guide you through fixing any issues before we prepare the final GitHub submission.

working

Great, Ankush! If `python manage.py check` and `python manage.py test` are both working, let's move on to the next stage.

Your application is now ready for a final quality review. We still need to make sure the tests cover the important scenarios and that your submission demonstrates your own understanding of the implementation.

Stage 16: Final submission preparation

Step 1: Test the application in your browser

Run:

Bash

```
python manage.py runserver
```

Open your application in the browser and verify:

- You can access the recommendation page.
- A valid order returns the recommended box.
- An order with no suitable box is handled correctly.
- An invalid order ID produces a 404 response.

Step 2: Prepare your GitHub repository

Make sure your repository contains:

```
box-selection-system/  
|  
├─ manage.py  
├─ requirements.txt  
├─ README.md  
├─ AI_USAGE.md  
├─ TEST_OUTPUT.md  
├─ .gitignore  
|  
├─ your_app/  
|   ├── models.py  
|   ├── views.py  
|   ├── urls.py  
|   ├── tests.py  
|   └── templates/  
|  
└─ your_project/  
    ├── settings.py  
    └── urls.py
```

This is an example structure. Keep your actual Django project and app names.

Remember to exclude your virtual environment, secret keys, and other sensitive configuration from GitHub.

Step 3: Prepare your documentation

Complete these files:

- `README.md` — project overview, installation, features, and usage.
- `AI_USAGE.md` — an honest record of your AI assistance and verification.
- `TEST_OUTPUT.md` — the actual output from your test run.

Export the actual chat transcript yourself, and write your own learning reflection as required by the assignment.

Stage 17: Let's review your code

Before you submit, I recommend one final review of your implementation.

Upload your completed Django project as a ZIP file [here](#).

I'll review:

1. Your models and relationships.
2. Your weight calculation.
3. Your dimensional fitting logic.

4. Your recommendation algorithm.
5. Your views, URLs, and templates.
6. Your automated tests.
7. Whether your implementation addresses the assignment requirements and documents its assumptions.

That will let us identify any remaining issues based on your actual code instead of guessing.

Next action: Upload your project ZIP.

ChatGPT is AI and can make mistakes.